Frontend API Integration Reference

This document outlines the API contracts for the nextBooq platform. Frontend developers must adhere strictly to these payloads and parameters.

Global Rules for Frontend Devs

1. **Base URL:** All API calls should be prefixed with `NEXT_PUBLIC_API_URL` (e.g., `https://api.nextbooq.com`).
2. **Authentication:** The backend expects authentication. Ensure your global Axios instance (`src/hooks/api/axios-instance.ts`) is configured to send credentials (if using HttpOnly cookies) or attaches the `Authorization: Bearer <token>` header to every protected request.
3. **Timezones:** All timestamps (`start_time`, `end_time`) must be sent and received in **UTC ISO 8601 string format**.

1. Authentication Module (`src/hooks/auth/`)

`POST /api/v1/auth/register`

- **Hook:** `useRegister()`
- **Purpose:** Register a new user.
- **Auth Required:** No
- **Body (JSON):**
  - `name` (string, required)
  - `email` (string, required)
  - `phone` (string, required)
  - `password` (string, required)
- **Response (201 Created):** Returns the JWT token and user profile `{ id, name, email, phone }`.

`POST /api/v1/auth/login`

- **Hook:** `useLogin()`
- **Purpose:** Authenticate a user.
- **Auth Required:** No
- **Body (JSON):**
  - `email` (string, required)
  - `password` (string, required)
- **Response (200 OK):** Returns the JWT token and user profile, including the `staff_memberships` array used for frontend RBAC (Role-Based Access Control).

2. Public Customer Module (`src/hooks/public/`)

`GET /api/v1/cafes/slug/:slug`

- **Hook:** `useCafeDetails(slug)`
- **Purpose:** Fetches branding and rules for the tenant based on the URL. Called in the Next.js Server Component `layout.tsx`.
- **Auth Required:** No
- **Path Params:** `slug` (e.g., "centralperk")
- **Response (200 OK):** The `Cafe` document (name, rules, timezone, etc.).

GET `/api/v1/cafes/:cafeId/available-slots`

- **Hook:** `useAvailability(cafeId, date, partySize)`
- **Purpose:** The core availability engine.
- **Auth Required:** No
- **Query Params:**
  - `date` (string, required): `YYYY-MM-DD`
  - `time` (string, required): `HH:mm` (24-hour)
  - `partySize` (number, required): > 0
  - `durationMinutes` (number, optional): Defaults to cafe settings.
- **Response (200 OK):** `{ available: boolean, slots: [...] }`

POST `/api/v1/cafes/:cafeId/reservations`

- **Hook:** `useBookTable()`
- **Purpose:** Creates a HOLD and generates a Razorpay order.
- **Auth Required:** Yes
- **Body (JSON):**
  - `start_time` (string, required): ISO Date String
  - `party_size` (number, required)
  - `duration_minutes` (number, optional)
  - `deposit_amount` (number, optional)
- **Response (201 Created):** `{ reservation_id, status: "PENDING_PAYMENT", razorpay_order_id, amount, currency, hold_expires_at }`

3. Host Dashboard Module (`src/hooks/host/`)

GET `/api/v1/cafes/:cafeId/reservations`

- **Hook:** `useManifest(cafeId, date)`
- **Purpose:** Fetch the daily schedule for the host stand.
- **Auth Required:** Yes (Requires `view_reservations` permission)
- **Query Params:** `date` (string, optional - `YYYY-MM-DD`).
- **Response (200 OK):** Array of reservations populated with guest contact details and assigned `table_numbers`.

PUT /api/v1/cafes/:cafeId/reservations/:resId/status

- **Hook:** `useUpdateStatus()`
- **Purpose:** Move a booking through its lifecycle (e.g., SEATED).
- **Auth Required:** Yes (Requires `edit_reservations` permission)
- **Body (JSON):**
  - `status` (string, required): CHECKED_IN, SEATED, COMPLETED, CANCELLED, NO_SHOW
  - `version` (number, required): The `__v` token from the reservation object. **CRITICAL for Optimistic Concurrency Control.**
- **Response (200 OK):** Updated reservation. *(Note: If you receive a 409 Conflict, it means the version token mismatched. Trigger a React Query invalidation to refresh the UI).*

4. Admin Inventory Module (`src/hooks/admin/`)

POST /api/v1/cafes

- **Hook:** `useCreateCafe()`
- **Purpose:** Onboarding wizard endpoint to provision a new tenant.
- **Auth Required:** Yes
- **Body (JSON):**
  - `org_name` (string, required)
  - `cafe_name` (string, required)
  - `slug` (string, required)
  - `timezone` (string, optional)
  - `reservation_rules` (object, optional)
- **Response (201 Created):** Created Organization and Cafe documents.

GET /api/v1/cafes/:cafeId/tables

- **Hook:** `useTables(cafeId)`
- **Purpose:** Fetch physical floorplan data.
- **Auth Required:** Yes (Requires `edit_tables` permission)
- **Response (200 OK):** Array of all active tables.

POST /api/v1/cafes/:cafeId/tables

- **Hook:** `useCreateTable()`
- **Purpose:** Create a new physical table.
- **Auth Required:** Yes
- **Body (JSON):**
  - `table_number` (string, required)
  - `capacity` (number, required)
  - `can_merge_with` (array of strings, optional): Adjacent table IDs.
  - `status` (string, optional): 'active' or 'out_of_service'

- **Response (201 Created):** New table document.

PUT `/api/v1/cafes/:cafeId/tables/:tableId`

- **Hook:** `useEditTable()`
- **Purpose:** Modify table details or adjacent merge links.
- **Auth Required:** Yes
- **Body (JSON):**
  - `table_number` (string, optional)
  - `capacity` (number, optional)
  - `can_merge_with` (array of strings, optional)
  - `status` (string, optional)
  - `is_deleted` (boolean, optional): Soft delete trigger.
- **Response (200 OK):** Updated table document.

5. Webhooks (For Information Only)

Frontend developers do not call this endpoint. It is triggered by Razorpay.

POST `/api/v1/webhooks/razorpay`

- **Purpose:** Transitions a reservation from `PENDING_PAYMENT` to `CONFIRMED`.
- **Action:** When a customer completes checkout on the frontend, the frontend should listen to the WebSocket or poll the reservation status until this webhook successfully updates the database.